



SCHOOL OF ADVANCED TECHNOLOGY

SAT301 FINAL YEAR PROJECT

**Design and Implementation of a React-Based Web
Workstation for PCB Defect Detection**

Final Thesis

In Partial Fulfillment
of the Requirements for the Degree of
Bachelor of Engineering

Student Name: Jingrui Wang

Student ID: 2142463

Supervisor: [Jingrui Wang]

Assessor: [Pengfei Song]

Abstract

Printed circuit boards are core components in electronic products, and small surface defects can affect product reliability, manufacturing yield, and maintenance cost. Traditional manual inspection is repetitive, subjective, and difficult to scale when images, video streams, and camera inputs must be reviewed continuously. This project focuses on the Web user interface part of a PCB defect detection system and designs a browser-based inspection workstation around an existing Python and YOLO detection backend. The aim is to improve usability, demonstration quality, and workflow continuity by providing a unified Web interface for operating the detection system.

The workstation is implemented with React 18, TypeScript, and Vite. It communicates with a Python backend through HTTP APIs and supports four input modes: single-image detection, batch image detection, video detection, and camera-frame detection. The interface also provides model selection, CPU/GPU device switching, confidence threshold adjustment, IoU threshold adjustment, label display control, original-image display control, result preview, detection table filtering, and result export. The backend uses OpenCV, Ultralytics YOLO, Torch, and session-based file management to process uploaded data and return structured JSON responses and media URLs. The project dataset contains six PCB defect categories: missing hole, mouse bite, open circuit, short, spur, and spurious copper. The recorded final training epoch achieved a precision of 0.95914, recall of 0.88724, mAP@0.5 of 0.92431, and mAP@0.5:0.95 of 0.46904.

The implemented workstation extends the original local inspection capability into a more accessible browser-based platform. The result demonstrates that React frontend engineering can add practical value to intelligent inspection systems by improving interaction design, parameter control, result visibility, and multi-scenario operation without replacing the underlying detection model.

Keywords: PCB defect detection; React; TypeScript; Web workstation; YOLO; human-computer interaction

Acknowledgements

I would like to express my sincere gratitude to my supervisor for the guidance, advice, and encouragement provided throughout the development of this final year project. The

supervisor's feedback helped me clarify the project scope, improve the structure of the system, and better understand how to present the technical work in an academic dissertation.

I would also like to thank the School of Advanced Technology for providing the academic environment and learning resources that supported this project. The knowledge gained from previous modules in software engineering, Web development, computer vision, and project management provided an important foundation for the design and implementation of this work.

This project also benefited from the availability of open-source technologies and documentation. Tools and libraries such as React, TypeScript, Vite, Python, OpenCV, Torch, and Ultralytics YOLO provided the technical basis for building and integrating the Web-based PCB defect detection workstation. Their documentation helped me understand implementation details and solve practical development problems during the project.

Finally, I would like to thank my family and friends for their patience and support during the completion of this dissertation. Their encouragement helped me maintain progress during the development, testing, writing, and revision stages of the project.

Contents

Abstract	2
Acknowledgements	2
Contents	3
List of Tables	4
List of Figures	4
List of Acronyms	5
Chapter 1 Introduction	5
1.1 Motivation	5
1.2 Problem Statement	7
1.3 Aim and Objectives	8
1.4 Project Scope	10
1.5 Main Contributions	11
1.6 Dissertation Structure	12
Chapter 2 Literature Review and Technical Background	13
2.1 PCB Defect Detection	13
2.2 Object Detection and YOLO	15
2.3 Web Interfaces for Inspection Systems	16
2.4 React, TypeScript, and Vite	18
2.5 Literature Gap and Project Direction	19
Chapter 3 Requirements and System Design	20
3.1 Functional Requirements	20
3.2 Non-Functional Requirements	22
3.3 System Architecture	23

3.4	User Interface Design	24
3.5	Data Model Design	26
3.6	API Design	27
3.7	Ethical, Professional, and Sustainability Considerations	29
Chapter 4	Implementation	30
4.1	Development Environment	30
4.2	Frontend Source Structure	31
4.3	API Layer Implementation	32
4.4	State Management and React Hooks	33
4.5	Single-Image Detection Mode	35
4.6	Batch Image Detection Mode	36
4.7	Video Detection Mode	37
4.8	Camera-Frame Detection Mode	38
4.9	Settings and Model Control	39
4.10	Result Visualisation and Export	40
4.11	Build and Deployment	42
Chapter 5	Testing and Evaluation	43
5.1	Testing Strategy	43
5.2	Dataset and Defect Categories	44
5.3	Functional Testing	45
5.4	Model Validation Results	49
5.5	Interface and Workflow Evaluation	50
5.6	Technical Evaluation	51
5.7	Limitations	52
Chapter 6	Conclusion and Future Work	53
6.1	Conclusion	54
6.2	Reflection	55
6.3	Future Work	56
Chapter 7	References	57
Chapter 8	Appendix A. Operation Guide	58
8.1	Appendix A.1 Development Mode	58
8.2	Appendix A.2 Production Mode	58
Chapter 9	Appendix B. API Summary	59

List of Tables

Table 3.1 Functional requirements of the Web workstation

Table 3.2 Core backend API endpoints

Table 5.1 Functional test cases

Table 5.2 Recorded model validation results

List of Figures

Figure 3.2 Overall React Web workstation interface

Figure 5.1 Single-image PCB defect detection result in the React Web workstation

Figure 5.2 Batch image detection result in the React Web workstation

Figure 5.3 Video inspection interface in the React Web workstation

Figure 5.4 Camera-frame detection interface in the React Web workstation

List of Acronyms

API Application Programming Interface

CPU Central Processing Unit

GPU Graphics Processing Unit

HTTP Hypertext Transfer Protocol

IoU Intersection over Union

mAP mean Average Precision

PCB Printed Circuit Board

SPA Single Page Application

UI User Interface

YOLO You Only Look Once

Chapter 1 Introduction

1.1 Motivation

In relation to motivation, a more detailed academic interpretation is required because the project connects intelligent inspection algorithms with the practical need for a usable and repeatable operator-facing environment. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design

decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that motivation has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

The motivation for this project is also connected to the broader movement from isolated algorithmic prototypes toward integrated intelligent inspection environments. In a manufacturing context, a detection algorithm is valuable only when it can be operated reliably by users who need to make inspection decisions. A Web workstation provides this operational layer by translating model inference into a sequence of observable actions: selecting an input, configuring parameters, receiving feedback, reviewing visual evidence, and preserving outputs for later analysis. This makes the project more than a user interface redesign; it is an attempt to bridge the gap between computer vision capability and practical software use.

The choice of a browser-based interface is particularly relevant because Web applications are increasingly used as lightweight front ends for local or networked engineering tools. A browser interface reduces dependence on a specific desktop GUI framework and makes the workflow easier to demonstrate across different machines. In this project, React is used not simply to create a visually modern page, but to organise an inspection task that contains multiple states, asynchronous operations, media previews, and user-controlled parameters. These requirements make the work suitable for a frontend engineering study within an intelligent inspection scenario.

Printed circuit boards are widely used in electronic devices, industrial systems, consumer products, communication equipment, and embedded computing platforms. Their manufacturing quality has a direct influence on the reliability and safety of the final product. A defect on a PCB surface can cause electrical instability, short circuits, open circuits, or long-term failure. Because PCB production is normally repetitive and image-based, visual inspection is a natural application area for computer vision and machine learning.

Manual PCB inspection is still useful in small-scale or final-review scenarios, but it has clear limitations. Human inspectors may become tired, inconsistent, or slow when many images must be reviewed. The decision process can also vary between inspectors, especially for small defects or borderline cases. Image-processing approaches and deep learning detectors can reduce this burden by automatically locating suspected defects and presenting them to the user for confirmation. However, an accurate model alone is not sufficient for a practical inspection system. Users also need a clear way to upload data, adjust parameters, preview results, compare outputs, and save evidence.

The project behind this dissertation already includes PCB defect detection capability. The main contribution discussed in this thesis is the design and implementation of a React-based Web workstation that exposes those capabilities through a browser interface. The Web interface is valuable because it improves accessibility, presentation quality, and workflow continuity. Instead of using separate scripts for images, videos, cameras, and result management, the user can operate the system through a single coordinated interface.

1.2 Problem Statement

In relation to problem Statement, a more detailed academic interpretation is required because the main difficulty is not only defect recognition, but the conversion of recognition into a coherent workflow that can be operated, inspected, and documented. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that problem Statement has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model,

settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

A key difficulty in many detection projects is that the model output is technically correct but operationally incomplete. For example, a script may produce an annotated image, but it may not clearly expose the active model, threshold configuration, device selection, latency, or result storage location. This creates a reproducibility problem because the user cannot easily determine the conditions under which a result was generated. The Web workstation addresses this problem by making runtime configuration and detection output visible in the same interface, thereby strengthening both usability and traceability.

The problem can therefore be framed as an integration and interaction design challenge. The system must accept heterogeneous inputs such as images, folders, videos, and camera frames, while presenting all outputs through a consistent visual structure. At the same time, it must avoid unnecessary complexity that would make local deployment difficult. This balance between functionality, simplicity, and maintainability is central to the implementation decisions described in this dissertation.

The core problem addressed by this project is the lack of a unified Web-based user interface for a PCB defect detection workflow. A detection model may identify defects, but users still need to manage several practical tasks: choosing a model, selecting a runtime device, adjusting confidence and IoU thresholds, uploading different input formats, viewing annotated images, filtering detection records, and exporting results. If these operations are fragmented across command-line scripts or a local desktop-only interface, the system becomes harder to demonstrate, maintain, and extend.

The project therefore focuses on the following question: how can an existing PCB defect detection capability be extended into a Web workstation that supports multi-mode input, parameter configuration, real-time feedback, result visualisation, and result export? The solution must remain lightweight enough for local deployment and coursework demonstration, while also following sound frontend engineering practices.

1.3 Aim and Objectives

In relation to aim and Objectives, a more detailed academic interpretation is required because the objectives translate a broad software problem into measurable engineering outcomes that can be implemented and tested. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem

context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that aim and Objectives has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

The objectives are intentionally expressed in engineering terms because the success of the project depends on implemented behaviour rather than conceptual design alone. Each objective corresponds to an observable system capability: the user can change settings, upload data, initiate detection, inspect results, and save outputs. This makes the project suitable for functional verification because the objectives can be tested through direct interaction with the Web workstation.

A further objective is to maintain a clear separation between frontend responsibilities and backend responsibilities. The React frontend should not contain model inference logic, while the Python backend should not determine the structure of the user interface. This separation encourages modularity and allows future work to improve the model, the backend service, or the interface independently. Such separation is a standard software engineering principle and is especially useful for final year projects that may later be extended by other developers.

The aim of this project is to design and implement a browser-based PCB defect detection workstation using React, TypeScript, and Vite, connected to a Python detection backend through HTTP APIs.

The project objectives are:

- To analyse the functional requirements of a Web interface for PCB defect detection.

- To design a single-page workstation layout that separates mode navigation, detection workspace, and parameter settings.

- To implement single-image, batch-image, video, and camera-frame detection workflows in React.

To integrate the frontend with a Python backend through typed API calls and structured JSON responses.

To provide model selection, device switching, confidence threshold adjustment, IoU threshold adjustment, and visual display switches.

To visualise detection outputs using preview images, video frame thumbnails, detection statistics, and searchable detection tables.

To support result saving through session-based export.

To evaluate the implemented system through functional testing, integration testing, and recorded model validation metrics.

1.4 Project Scope

In relation to project Scope, a more detailed academic interpretation is required because the scope defines the project as a Web workstation contribution while still acknowledging the detection backend as the technical context. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that project Scope has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

The scope limitation is deliberate and important. If the project attempted to make equal contributions in model training, backend engineering, frontend design, and user evaluation, the depth of each part would be reduced. By focusing the dissertation on the React Web workstation, the implementation can be evaluated according to interface completeness,

workflow design, typed data modelling, asynchronous communication, and integration quality. These areas are directly aligned with the submitted source code and screenshots.

Nevertheless, the project cannot be separated entirely from the detection model because the interface is meaningful only when connected to a real inference pipeline. For this reason, the thesis includes model classes, validation metrics, and backend behaviour as contextual evidence. These elements support the evaluation of the Web system without shifting the main claim of the dissertation away from frontend implementation and user-facing software design. The scope of this thesis is the Web workstation implementation and its integration with the detection backend. The project uses an existing YOLO-based detection capability and trained model files. Therefore, the primary engineering contribution is not the invention of a new object detection algorithm. Instead, the contribution is the transformation of the detection capability into a usable Web-based system that supports multiple operating modes and a complete user workflow.

The thesis discusses the dataset and model metrics because they are relevant to the system context. However, the main design and implementation analysis focuses on the React frontend, the API contract, the backend integration strategy, and the user-facing inspection workflow.

1.5 Main Contributions

In relation to main Contributions, a more detailed academic interpretation is required because the contribution lies in system integration, interaction design, multi-mode inspection support, and traceable result handling. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that main Contributions has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly

important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

The contributions can also be interpreted through the lens of human-computer interaction. The workstation reduces cognitive effort by placing related controls and feedback in stable regions of the page. Users do not need to remember command-line options or file naming conventions while testing different modes. Instead, they can interact with clearly labelled controls and immediately observe how the system responds. This contributes to operational clarity, which is a critical quality for inspection software.

Another contribution is the use of a common result representation across different input modes. Whether the source is a still image, a batch item, a sampled video frame, or a camera frame, the frontend displays class information, confidence, and coordinates through the same table structure. This consistency reduces implementation duplication and improves user understanding, because the meaning of a result row does not change between modes.

The project makes four main contributions. First, it provides a single-page React workstation for PCB defect detection, replacing fragmented operation with a unified interface. Second, it supports four practical input scenarios: single-image, batch-image, video, and camera-frame detection. Third, it exposes runtime configuration through Web controls, including model selection, CPU/GPU choice, confidence threshold, IoU threshold, and display options. Fourth, it implements result visualisation and export through annotated media, detection tables, thumbnails, summaries, and session-based saving.

These contributions are important because an intelligent inspection system must be usable as well as technically capable. The Web workstation makes the detection process more visible, repeatable, and suitable for demonstration or future deployment.

1.6 Dissertation Structure

In relation to dissertation Structure, a more detailed academic interpretation is required because the dissertation structure supports a logical movement from problem context to implementation evidence and critical evaluation. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable

manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that dissertation Structure has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

The structure of the dissertation follows a design-oriented engineering narrative. The early chapters explain the problem context and justify the need for a Web workstation. The middle chapters describe requirements, architecture, and implementation decisions. The evaluation chapter then relates those decisions to observed system behaviour. This organisation supports a coherent argument: the project identifies a usability and integration gap, implements a suitable Web solution, and verifies the solution through functional evidence.

Chapter 1 introduces the motivation, problem statement, aim, objectives, scope, and contributions. Chapter 2 reviews PCB defect detection, object detection, Web-based inspection workstations, and frontend technologies. Chapter 3 presents the requirements and design of the system. Chapter 4 explains the implementation of the React frontend and its integration with the Python backend. Chapter 5 evaluates the system through tests, metrics, and limitations. Chapter 6 concludes the project and proposes future improvements.

Chapter 2 Literature Review and Technical Background

2.1 PCB Defect Detection

In relation to pCB Defect Detection, a more detailed academic interpretation is required because PCB inspection combines small visual targets, manufacturing reliability concerns, and the requirement for interpretable localisation output. A final year project should not only describe what was implemented, but should also explain why the implementation is

appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that pCB Defect Detection has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

PCB defect detection is a demanding visual task because many defects are small, localised, and visually similar to normal board patterns. Copper traces, solder pads, drilled holes, and background texture can create complex visual scenes. A defect such as a missing hole or spur may occupy only a small region of the image, yet it can still have significant electrical consequences. For this reason, a useful inspection system must provide both localisation and classification, allowing users to inspect where and why the model produced a warning.

From a system design perspective, the defect categories also influence interface requirements. A user must be able to distinguish between defect types and understand the confidence associated with each prediction. The interface should therefore avoid hiding raw detection information behind a single summary number. In this project, the detection table and annotated preview work together: the preview shows the spatial location of defects, while the table records the class, confidence, and coordinates. This dual representation is more suitable for inspection than either visual output or tabular output alone.

PCB defect detection has been studied for many years because PCB manufacturing combines high production volume with strict quality requirements. Earlier inspection approaches often used image subtraction, template matching, thresholding, morphology, and rule-based image processing. These techniques can work well when illumination, image alignment, board design, and defect appearance are controlled. However, they can become fragile when the input distribution changes.

Modern inspection systems increasingly use machine learning and deep learning to improve robustness. A detector can learn visual features from annotated samples rather than depending only on manually designed rules. This is particularly relevant for PCB defects because the appearance of small faults can vary across board designs, copper traces, holes, pads, and imaging conditions. The defect categories used in this project are missing hole, mouse bite, open circuit, short, spur, and spurious copper. These classes cover both structural absence and unwanted conductive material, so the Web interface must present both class labels and localisation boxes clearly.

The literature shows that the technical challenge is not only classification. Localisation is important because an operator must know where a suspected defect is located. A Web workstation therefore needs to show annotated images and bounding-box metadata rather than returning only a class name. This requirement directly influenced the design of the detection table, preview panel, and result export workflow.

2.2 Object Detection and YOLO

In relation to object Detection and YOLO, a more detailed academic interpretation is required because object detection provides both class information and spatial localisation, which makes it suitable for visual inspection workflows. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that object Detection and YOLO has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

YOLO-based object detection is appropriate for an interactive inspection workstation because it is designed for efficient inference and direct bounding-box prediction. Although the Web frontend does not run the neural network, the interface must still be designed around the characteristics of object detection. Unlike simple image classification, detection can return multiple objects per image. This requires the frontend to handle variable-length result arrays, visual previews, and row-based inspection of individual detections.

The confidence and IoU thresholds exposed in the settings panel reflect common object detection trade-offs. A lower confidence threshold may reveal more potential defects but may also increase false positives. A higher threshold may make results cleaner but may miss subtle defects. Similarly, IoU behaviour affects how overlapping predictions are filtered. By placing these controls in the interface, the system allows the user to observe detection sensitivity and refine inspection behaviour without changing backend code.

Object detection combines classification and localisation. A detector predicts both the category of an object and its position in the image, typically using bounding boxes. The YOLO family is widely known for real-time detection because it treats detection as a single regression problem over image regions [3]. This property makes YOLO-style detection suitable for interactive systems where users expect quick feedback after uploading an image or video.

In this project, the backend uses Ultralytics YOLO through Python. The frontend does not directly execute the neural network. Instead, it sends files or camera frames to the backend, then receives structured detection results. This separation is important. It keeps the browser interface lightweight, avoids exposing model execution details to the frontend, and allows Python libraries such as OpenCV, Torch, and Ultralytics to remain on the backend side.

The Web workstation uses model outputs in a practical form. For each detection it displays the class key, class label, confidence score, and bounding box coordinates. This mapping from model inference to readable inspection data is essential because users need to interpret the result, not merely know that the model has run.

2.3 Web Interfaces for Inspection Systems

In relation to web Interfaces for Inspection Systems, a more detailed academic interpretation is required because a Web interface can improve accessibility, demonstration value, and operational consistency for intelligent inspection tools. A final year project should not only describe what was implemented, but should also explain why the implementation is

appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that web Interfaces for Inspection Systems has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

A Web interface is not merely a convenience layer; it shapes how users reason about inspection outcomes. In a desktop or script-based workflow, the user may focus on one file at a time and manually search output folders. In a workstation layout, inputs, settings, outputs, and status messages are visible together. This helps users maintain context while switching between operating modes. It also makes the system easier to present during assessment because the interface communicates the capabilities of the project directly.

The inspection domain also requires a careful balance between visual density and readability. Too little information would make the system look incomplete, but excessive visual complexity would make the result hard to interpret. The implemented interface uses cards, panels, summary tiles, and tables to organise information into predictable zones. This design supports scanning and repeated use, which is more appropriate for an engineering workstation than a marketing-style page.

An inspection system normally has two audiences. Technical users may care about models, thresholds, devices, and API behaviour. Non-technical users may care more about whether the system is easy to operate and whether the result can be understood visually. A Web interface can support both groups if it organises the workflow clearly. Browser-based interfaces also simplify demonstration because they can be opened without requiring the user to understand Python scripts or desktop GUI dependencies.

For PCB inspection, a useful interface should support input selection, processing feedback, result preview, result comparison, and export. It should also expose model parameters in a

controlled way. If threshold settings are hidden, users cannot experiment with false positives and false negatives. If the result display is weak, users cannot evaluate whether the detector is actually useful. These observations motivated the workstation layout used in this project: a left navigation rail, a central workspace, and a right parameter panel.

2.4 React, TypeScript, and Vite

In relation to react, TypeScript, and Vite, a more detailed academic interpretation is required because the selected frontend stack supports stateful interaction, typed API contracts, and efficient development and deployment. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that react, TypeScript, and Vite has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

React is suitable for this project because the interface is highly state-dependent. The active mode determines which toolbar is shown, which upload input is available, which result preview is displayed, and which save action is enabled. These conditional interface states map naturally to React component state. The use of hooks also keeps the implementation readable by keeping related state and behaviour near the component that uses it.

TypeScript contributes to reliability by making the structure of backend responses explicit. In a detection application, response fields such as ``sessionId``, ``previewUrl``, ``detections``, and ``summary`` are not optional details; they determine what the user can see and what actions are available. Typed interfaces reduce the risk of using a field incorrectly and provide clearer

documentation for future maintainers. Vite complements this by offering a fast development cycle and a straightforward production build path.

React is a component-based JavaScript library for building user interfaces. Its state and rendering model is suitable for interactive pages where user actions change what should be displayed. The implemented workstation uses React hooks such as `'useState'`, `'useEffect'`, and `'useRef'`. It also uses React 18 features such as `'startTransition'` and `'useDeferredValue'` to keep result rendering and table filtering responsive when the interface is updated [6].

TypeScript adds static type checking to JavaScript. This is useful in a detection workstation because the frontend exchanges structured data with the backend. The project defines types for detection settings, runtime status, model options, image results, batch results, video results, camera results, and detection items. These types reduce ambiguity in the API contract and make the implementation easier to maintain [7].

Vite is used as the frontend development and build tool. During development it runs the Web interface on `'127.0.0.1:5173'` and proxies `'/api'` and `'/media'` requests to the Python backend on `'127.0.0.1:8765'`. In production, the frontend is built into static assets under `'web-ui/dist'`, and the Python backend can serve the built files directly. This approach is suitable for a local demonstration project because it does not require a separate Node server after building [8].

2.5 Literature Gap and Project Direction

In relation to literature Gap and Project Direction, a more detailed academic interpretation is required because the project addresses the gap between model-centred detection work and complete user-facing inspection systems. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that literature Gap and Project Direction has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent

interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

The project direction is therefore positioned between computer vision research and applied software engineering. Many studies focus on model accuracy, while many frontend projects focus on general interface construction. This dissertation combines the two perspectives by asking how a trained detector can be made operationally usable through a carefully designed Web interface. The novelty is not a new detection architecture, but the system-level integration of inference, configuration, visualisation, and result management.

The reviewed background suggests a gap between model-centred detection projects and user-centred inspection tools. Many projects demonstrate detection accuracy, but a practical inspection workflow also requires interaction design, parameter control, result traceability, and support for multiple input sources. This project addresses that gap by building a Web workstation around an existing detection backend. The engineering focus is therefore the integration of computer vision output into a browser-based workflow that users can operate, inspect, and extend.

Chapter 3 Requirements and System Design

3.1 Functional Requirements

In relation to functional Requirements, a more detailed academic interpretation is required because the functional requirements define the observable behaviours needed for a practical PCB inspection workstation. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that functional Requirements has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the

user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

The functional requirements define the minimum behaviour required for the workstation to be useful in practice. They are also designed to cover different inspection contexts. Single-image detection supports quick verification, batch detection supports offline review, video detection supports continuous visual records, and camera-frame detection supports near-real-time demonstrations. This range of inputs shows that the frontend is not a static wrapper around one API call, but a multi-mode workstation.

Each requirement also implies a corresponding interface state. For example, batch detection requires file ordering, selected item tracking, thumbnail browsing, and aggregate target counts. Video detection requires a distinction between the annotated output video and sampled frame previews. Camera detection requires media stream management and periodic frame upload. Identifying these states in the requirements stage helped shape the React implementation and prevented the interface from becoming an unstructured collection of buttons.

The functional requirements were derived from the existing desktop detection capability and from the need to provide a browser-based inspection workflow. The Web workstation must cover the main detection scenarios and expose enough controls for meaningful use.

Table 3.1: Functional requirements of the Web workstation

ID	Requirement	Description
FR1	Single-image detection	The user can upload one PCB image and receive an annotated result.
FR2	Batch image detection	The user can upload a folder or multiple images and browse each result.
FR3	Video detection	The user can upload a video, receive an annotated output video, and inspect sampled frames.
FR4	Camera-frame detection	The user can enable the browser camera and periodically send captured frames for detection.
FR5	Model selection	The user can select an available `.pt`, `.pth`, or `.onnx` model.
FR6	Parameter	The user can change confidence threshold, IoU threshold,

	adjustment	display labels, and original image mode.
FR7	Device switching	The user can choose CPU or GPU, with GPU disabled when unavailable.
FR8	Result visualisation	The user can view previews, target counts, latency, confidence scores, and bounding boxes.
FR9	Result filtering	The user can filter detection rows by class, confidence, or coordinate text.
FR10	Result export	The user can save session results into the `save_data` directory.

3.2 Non-Functional Requirements

In relation to non-Functional Requirements, a more detailed academic interpretation is required because qualities such as usability, maintainability, responsiveness, and traceability determine whether the system is practical beyond a simple prototype. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that non-Functional Requirements has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

Non-functional requirements are especially important for a user-facing inspection tool because poor responsiveness or unclear feedback can undermine trust in the system. If the user uploads a file and the interface gives no indication of progress or failure, the detection capability becomes difficult to use even if the backend is correct. The project therefore

includes busy states, status banners, disabled buttons, summary tiles, and runtime readiness indicators. These features contribute to perceived reliability.

Maintainability is another major non-functional requirement. The implementation avoids unnecessary dependencies and keeps the frontend API layer separate from the component logic. This means that if a backend endpoint changes, the adjustment can be made primarily in ``api.ts`` and ``types.ts``. Similarly, if the interface is redesigned later, the backend detection service does not need to be rewritten. This separation reduces long-term maintenance cost.

The system also needs to meet several non-functional requirements. Usability is important because the workstation is intended for demonstration and inspection, not only for developers. Responsiveness is important because detection results may contain multiple rows and preview images. Maintainability is required because the frontend and backend should evolve independently. Portability is required because the system should run locally on a normal development machine. Traceability is also required because inspection results should be saved for later review.

These non-functional requirements influenced the design. TypeScript types were used to make the API contract explicit. The frontend state is kept within the React application instead of adding unnecessary global state libraries. The backend returns structured JSON and stores media outputs in session folders. The frontend build is hosted by the Python backend in production mode, reducing deployment complexity.

3.3 System Architecture

In relation to system Architecture, a more detailed academic interpretation is required because the architecture separates browser-based interaction from Python-based detection, allowing each layer to perform its strongest role. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that system Architecture has direct consequences for maintainability and future extension. When a detection project is organised as a set of

unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

The architecture follows a client-service pattern. The browser client is responsible for interaction, presentation, and local media capture, while the Python backend is responsible for computationally intensive detection work. This division is appropriate because browser environments are well suited to user interaction but less suitable for running large Python-based computer vision pipelines. The backend can rely on mature Python libraries and trained model files, while the frontend remains lightweight.

The architecture also supports two deployment modes. During development, Vite provides fast frontend iteration and proxies API requests to the backend. In production, the built frontend can be served directly by the Python backend. This reduces the number of services required for local use. For a final year project, this is a pragmatic design because it allows the system to be demonstrated with minimal deployment overhead while still preserving a clear frontend-backend boundary.

The system is designed as a layered Web application. The browser runs the React frontend. The frontend sends HTTP requests to the Python backend. The backend manages settings, loads the detection model, runs inference, writes annotated media, and returns JSON responses. Media files are served through `/media` URLs.

Browser user interface -> React + TypeScript workstation -> Fetch API and FormData -> Python HTTP backend -> Detection service -> YOLO, OpenCV, Torch -> Session media and exported results

This design separates user interaction from model execution. The frontend is responsible for user workflow, interface state, and visualisation. The backend is responsible for computational work, file decoding, model loading, inference, drawing results, and session management. The separation makes the system easier to reason about and test.

3.4 User Interface Design

In relation to user Interface Design, a more detailed academic interpretation is required because the interface layout organises navigation, input, configuration, preview, and result

details into stable regions. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that user Interface Design has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

The interface layout reflects the order in which users make decisions. A user first selects a detection mode, then chooses or captures input, adjusts parameters if necessary, and reviews results. The left rail, central workspace, and right settings panel follow this workflow. This spatial consistency is valuable because users can change modes without relearning the interface. The settings remain on the right, the preview remains in the centre, and navigation remains on the left.

The visual design also supports assessment and documentation. Screenshots of the interface can show several important project features at once: the active mode, model name, runtime device, threshold values, preview output, and detection table. This makes the interface useful not only for operation but also for communicating the implemented system in the dissertation. The screenshots inserted into this document demonstrate how the Web workstation presents different detection scenarios.

The Web page is designed as a single-page workstation. The left rail contains the detection mode navigation. The central area contains the active detection workspace, preview panel, thumbnail rail, summary tiles, and detection table. The right panel contains settings for the model, runtime device, confidence threshold, IoU threshold, and display switches.

Mode selection -> File, folder, video, or camera input -> Backend detection request -> Result preview -> Detection list and statistics -> Optional export

This layout keeps the most important workflow elements visible at the same time. A user does not need to move between separate pages to change thresholds or inspect output. This is useful in an inspection context because users often adjust thresholds after seeing the result.

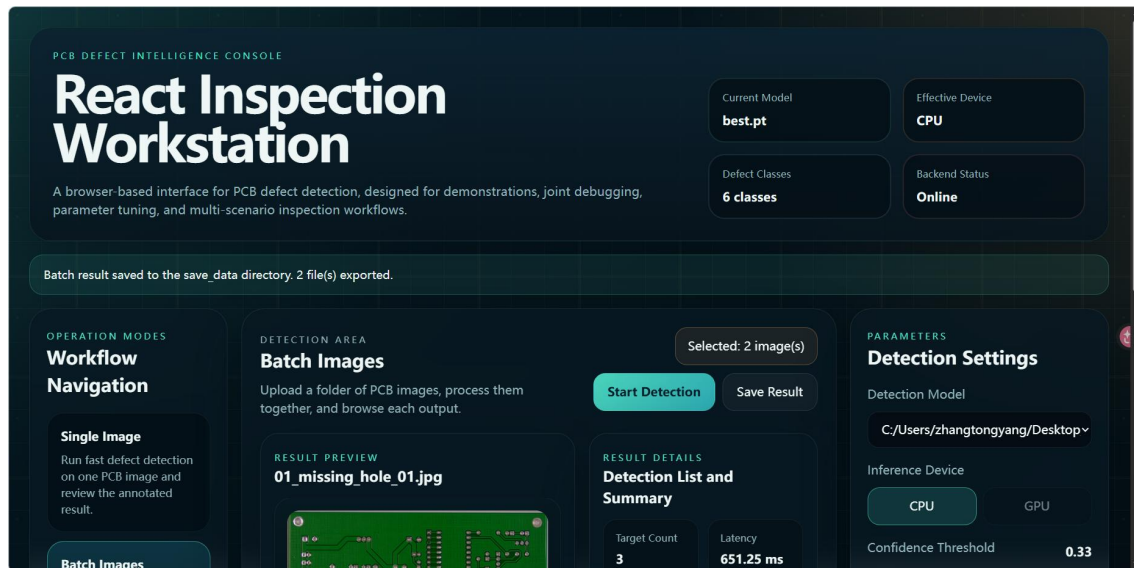


Figure 3.2 Overall React Web workstation interface

3.5 Data Model Design

In relation to data Model Design, a more detailed academic interpretation is required because the data model normalises outputs from image, batch, video, and camera modes into a shared frontend representation. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that data Model Design has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

The data model is designed to normalise different detection modes into a shared representation. Although image, batch, video, and camera inputs are operationally different, their detection rows have the same essential structure. This allows a single detection table component to display outputs from multiple modes. Such reuse is an important frontend engineering decision because it reduces duplicated rendering logic and helps maintain consistent presentation.

The settings data model also separates requested configuration from runtime capability. For example, the user may request GPU inference, but the backend must determine whether a GPU is actually available. Returning `'gpu_available'` and `'effective_device'` helps the frontend present accurate information. This prevents a misleading interface state in which the UI appears to use GPU while the backend is actually using CPU.

The frontend data model is defined in `'web-ui/src/types.ts'`. The most important type is `'DetectionItem'`, which contains the detection index, class ID, class key, class label, confidence value, formatted confidence text, and bounding box coordinates. This structure is shared by image, batch, video, and camera results.

The detection settings include model path, device, confidence threshold, IoU threshold, label display, original image display, GPU availability, and effective device. This allows the UI to show both the desired setting and the actual runtime behaviour. For example, GPU may be selected, but the effective device is CPU if CUDA is unavailable.

3.6 API Design

In relation to aPI Design, a more detailed academic interpretation is required because the API design defines the communication boundary between the React client and the Python detection service. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that aPI Design has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the

user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

The API design emphasises clarity rather than excessive abstraction. Each detection mode has a dedicated endpoint, which makes the request purpose explicit. At the same time, the responses follow similar patterns by returning a session identifier, preview media, detections, and summary information. This combination of endpoint specificity and response consistency makes the frontend code easier to understand.

The use of multipart form data is appropriate because the input files are binary media rather than simple JSON objects. The frontend wraps selected files in `FormData`, while the backend parses uploaded fields and converts the content into images or videos. This design uses standard Web mechanisms and avoids custom encoding. It also allows browser file inputs and camera-captured blobs to be handled through the same general upload pathway.

The frontend API layer is implemented in `web-ui/src/api.ts`. It uses the Fetch API and `FormData` for uploads. The backend returns JSON in a consistent format. Table 3.2 summarises the main endpoints.

Table 3.2: Core backend API endpoints

Endpoint	Method	Purpose
<code>/api/status</code>	GET	Returns backend status, dependency readiness, defect classes, and settings.
<code>/api/settings</code>	GET	Returns current detection settings.
<code>/api/settings</code>	PUT	Updates model path, device, thresholds, and display options.
<code>/api/models</code>	GET	Lists available model files and the current model.
<code>/api/detect/image</code>	POST	Detects defects in one uploaded image.
<code>/api/detect/batch-images</code>	POST	Detects defects in multiple uploaded images.
<code>/api/detect/video</code>	POST	Detects defects in an uploaded video and sampled frames.
<code>/api/detect/camera-frame</code>	POST	Detects defects in a browser camera frame.
<code>/api/sessions/{sessionId}/save</code>	POST	Exports saved session results to <code>save_data</code> .

3.7 Ethical, Professional, and Sustainability Considerations

In relation to ethical, Professional, and Sustainability Considerations, a more detailed academic interpretation is required because inspection software must expose uncertainty, support review, and avoid presenting automated output as unquestionable truth. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that ethical, Professional, and Sustainability Considerations has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

An intelligent inspection system should be presented as decision support rather than as an unquestionable authority. The interface supports this principle by displaying confidence values and visual evidence. Users can inspect the location and strength of each prediction rather than receiving only a binary pass-or-fail result. This is important in engineering contexts where incorrect acceptance or rejection of a component may have cost, quality, or safety implications.

Professional responsibility also includes transparency about limitations. The system depends on the representativeness of the dataset, the quality of the trained model, and the consistency of input images. The Web interface cannot remove these limitations, but it can make model behaviour more observable. By exposing thresholds and result details, the system encourages informed use rather than blind reliance on automated predictions.

The system is intended to support inspection rather than replace human responsibility completely. A detection model can produce false positives and false negatives, especially

when input images differ from the training data. Therefore, the interface presents confidence values and bounding boxes so that users can review the evidence.

From a professional perspective, the system should not hide uncertainty. Adjustable thresholds and visible result tables help users understand and control the behaviour of the detector. From a sustainability perspective, automated inspection can reduce repeated manual work and may reduce waste by detecting manufacturing issues earlier. However, GPU inference consumes energy, so the interface supports CPU/GPU awareness and should be extended in future work with performance and energy monitoring.

Chapter 4 Implementation

4.1 Development Environment

In relation to development Environment, a more detailed academic interpretation is required because the environment combines Web engineering tools with mature Python computer vision libraries. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that development Environment has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

The development environment was selected to support rapid iteration while keeping the project understandable. React and Vite provide a productive frontend workflow, while the Python backend allows direct reuse of the existing detection stack. This is a practical combination for a final year project because it avoids the need to port computer vision code

into JavaScript or WebAssembly. The frontend and backend can be developed and tested independently but still operate as one system.

The implementation also benefits from using widely adopted technologies. React, TypeScript, Python, OpenCV, Torch, and Ultralytics all have substantial documentation and community support. This reduces technical risk and makes the project easier to maintain. In an academic project, using established technologies is often preferable to using obscure tools because the design choices can be justified and reproduced by others.

The frontend is implemented using React 18, TypeScript, and Vite. The project configuration in `web-ui/package.json` defines scripts for development, build, and preview. The dependency set is intentionally small: `react` and `react-dom` are runtime dependencies, while TypeScript, Vite, the React plugin, and type packages are development dependencies.

The backend is implemented in Python. It uses the standard library `BaseHTTPRequestHandler` and `ThreadingHTTPServer` for HTTP service, rather than adding Flask or FastAPI. The detection service uses OpenCV for image and video handling, Ultralytics YOLO for model inference, Torch for device availability, NumPy for image arrays, and Pillow as a fallback for label drawing. This choice keeps the Web backend lightweight and close to the existing detection code.

4.2 Frontend Source Structure

In relation to frontend Source Structure, a more detailed academic interpretation is required because the source structure keeps the application understandable by separating rendering, API calls, and data types. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that frontend Source Structure has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly

important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

The compact frontend source structure reflects the focused scope of the workstation. `'App.tsx'` acts as the coordination point for interaction and state, while `'api.ts'` and `'types.ts'` isolate communication and data definitions. This separation is sufficient for the current scale and avoids the overhead of a larger application architecture. If the project later grows to include user accounts, history pages, or training dashboards, the same codebase could be refactored into smaller components and routes.

The current structure also makes the implementation easier to review. A reader can understand the main workflow by examining a small number of files. This is useful for dissertation assessment because the connection between written description and source code is direct. The implementation does not hide functionality in a complex framework layer, so the engineering decisions are visible.

The frontend source files are located under `'web-ui/src'`. The main files are `'main.tsx'`, `'app.tsx'`, `'api.ts'`, `'types.ts'`, and `'app.css'`.

`'main.tsx'` mounts the React application. `'app.tsx'` implements the workstation layout, state management, mode switching, camera handling, result rendering, and toolbar actions. `'api.ts'` encapsulates HTTP requests to the backend. `'types.ts'` defines TypeScript interfaces for settings, runtime status, models, detection items, and result payloads. `'app.css'` implements the visual style of the workstation.

This structure is appropriate for the project scale. The application is a focused single-page workstation, so adding React Router or a global state management library would increase complexity without clear benefit. The state is local to the main application component and directly connected to the workflow.

4.3 API Layer Implementation

In relation to API Layer Implementation, a more detailed academic interpretation is required because the API layer centralises request behaviour and makes network communication more maintainable. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to

operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that aPI Layer Implementation has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

The API layer improves code quality by centralising request behaviour. Without this layer, each button handler would need to repeat fetch calls, JSON parsing, error handling, and form-data construction. Centralisation reduces duplication and makes behaviour more consistent. For example, all failed responses pass through the same error mechanism, which allows the UI to display a clear banner message.

The layer also provides a typed boundary between the frontend and backend. Each exported function returns a specific TypeScript type, such as `'ImageResult'` or `'VideoResult'`. This design helps ensure that UI code handles the correct fields for each mode. It also documents the expected backend contract in code, which is useful when the backend is modified or extended.

The `'api.ts'` file defines a shared `'requestJson'` helper. This helper sends requests to the backend, parses the JSON response, and throws an error if the response is not successful. File uploads are handled by creating `'FormData'` objects and appending selected files under the field names expected by the backend.

The frontend exposes functions such as `'getStatus'`, `'getSettings'`, `'getModels'`, `'updateSettings'`, `'detectImage'`, `'detectBatch'`, `'detectVideo'`, `'detectCameraFrame'`, and `'saveSession'`. This creates a clean boundary between UI components and network details. The React component calls these functions without manually repeating request construction each time.

4.4 State Management and React Hooks

In relation to state Management and React Hooks, a more detailed academic interpretation is required because React hooks manage user interaction, asynchronous detection state, and browser resources such as camera streams. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that state Management and React Hooks has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

The decision to use React hooks rather than an external state management library is justified by the size and shape of the application. Most state is local to the workstation page and directly related to rendering. Introducing Redux or another global store would add indirection without solving a concrete problem. The hook-based approach keeps state transitions close to the event handlers and effects that use them.

The use of references is also important. Camera streams, interval identifiers, and canvas elements are mutable resources that should persist across renders but should not trigger rendering on every update. `useRef` is therefore a suitable mechanism. This distinction between state that affects the interface and references that manage imperative browser resources demonstrates an appropriate understanding of React behaviour.

The workstation uses `useState` for active mode, backend status, settings, model list, busy state, banner messages, selected files, and detection results. It uses `useRef` for objects that should persist without causing re-rendering, such as the folder input element, camera video element, canvas element, camera interval ID, media stream, and camera busy flag.

The application uses `useEffect` to bootstrap backend status and model information when the page loads. Another effect stops the camera when the user leaves the camera mode. This prevents the browser camera stream from continuing unexpectedly.

The project also uses `startTransition` when updating result data after detection. This marks large result rendering as a lower-priority update and helps maintain interface responsiveness. The detection table uses `useDeferredValue` for search text so that filtering does not interrupt typing when the result list is updated.

4.5 Single-Image Detection Mode

In relation to single-Image Detection Mode, a more detailed academic interpretation is required because single-image detection validates the simplest complete path from file upload to annotated output. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that single-Image Detection Mode has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

Single-image mode is the most direct representation of the inspection workflow. It provides a simple path from user input to model output and is therefore useful for validating the entire pipeline. When a user selects an image, the interface stores the file, enables the detection action, sends the file to the backend, and renders the returned preview and detection rows. This sequence verifies file upload, backend inference, media serving, and frontend rendering. The screenshot of the single-image result shows why this mode is important for the dissertation. It demonstrates that the Web interface is connected to real detection output

rather than being a static design mock-up. The user can see the selected PCB image, target count, latency, label configuration, and detection table. These elements provide concrete evidence that the React workstation performs an operational role in the system.

Single-image detection allows the user to choose one image file and start detection. The frontend sends the selected file to `/api/detect/image`. The backend decodes the image, runs YOLO inference, draws bounding boxes and labels if enabled, writes an annotated PNG image into the session directory, and returns a preview URL and detection list.

The UI then displays the preview image, target count, latency, label display setting, original image mode, and detection rows. The user can export the result through the save button. This mode is useful for quickly checking a single PCB image and validating model behaviour.

4.6 Batch Image Detection Mode

In relation to batch Image Detection Mode, a more detailed academic interpretation is required because batch detection extends the workflow to repeated inspection and comparison across multiple PCB images. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that batch Image Detection Mode has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

Batch mode introduces additional complexity because the user is no longer inspecting one result. The interface must support a collection of outputs, allow the user to switch between items, and preserve the relationship between each thumbnail and its source file. The

thumbnail rail addresses this by giving the user a visual index of processed images. This design is particularly suitable for offline inspection, where many PCB images may need to be reviewed after processing.

The batch screenshot demonstrates that the same result panel and detection table can be reused in a multi-image context. The selected image is shown in the preview, while thumbnails provide access to neighbouring outputs. This supports comparative review and shows that the frontend state model can handle both aggregate summaries and selected-item details. The implementation therefore extends beyond a single-file demo.

Batch image detection supports folder-style upload. The frontend sets the file input attributes ``webkitdirectory`` and ``directory`` so that a browser can select a folder. Files are sorted by relative path or file name before upload, which makes result browsing more predictable.

The backend processes each image in sequence, writes annotated outputs, and returns an item list. The frontend displays a thumbnail rail so that the user can select each image and inspect its detections. This mode is useful for offline review of many PCB images and for comparing detection performance across a dataset.

4.7 Video Detection Mode

In relation to video Detection Mode, a more detailed academic interpretation is required because video detection introduces temporal media processing and demonstrates support for heavier inspection inputs. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that video Detection Mode has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model,

settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

Video mode is relevant because inspection scenarios may involve moving production lines, recorded camera feeds, or continuous visual monitoring. A video file contains many frames, so the interface must avoid overwhelming the user. The implemented design separates the annotated video from sampled frame previews. This allows the user to play the processed video while also inspecting representative frames in a structured way.

From an engineering perspective, video detection tests the asynchronous nature of the system more strongly than still-image detection. Processing can take longer, the result includes both a media file and frame records, and the frontend must present sampled detections without blocking the rest of the interface. This makes video mode a useful demonstration of the workstation's ability to support heavier media workflows.

Video detection allows the user to upload a video file. The backend stores the video temporarily, opens it with OpenCV, reads frames, runs detection on each frame, writes an annotated output video, and saves sampled frame previews at a regular interval. The frontend displays the video and a frame thumbnail rail. For each sampled frame, it displays the detections and summary information.

This mode is more computationally expensive than image detection because each frame requires model inference. The interface therefore displays processing feedback and separates the annotated video from sampled frame inspection. This design gives the user both a continuous output and a way to inspect representative frames.

4.8 Camera-Frame Detection Mode

In relation to camera-Frame Detection Mode, a more detailed academic interpretation is required because camera-frame detection links browser media capture with backend inference in a near-real-time workflow. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that camera-Frame Detection Mode has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

Camera-frame detection demonstrates the connection between browser-native media APIs and backend computer vision. The frontend obtains a camera stream, captures frames through a canvas, converts each frame to an uploadable file, and sends it to the backend. This workflow illustrates how a Web interface can act as a bridge between local hardware and Python-based inference.

The camera mode also raises practical interaction issues. The user must grant browser permission, the stream must be stopped when the mode changes, and frame uploads must not overlap excessively. The implementation addresses these concerns through camera state, cleanup effects, and a busy flag. These details are important because camera features can easily become unstable if resource management is ignored.

The camera mode uses the browser `MediaDevices.getUserMedia` API to access a camera. Once the stream is active, the frontend draws the current video frame onto a hidden canvas at a fixed interval. The canvas content is converted into a PNG blob, wrapped as a `File`, and uploaded to `/api/detect/camera-frame`.

The backend treats the uploaded frame as an image and returns an annotated result as a base64 data URL. The frontend displays the returned image and detection list. A busy flag prevents overlapping requests when a previous frame is still being processed. This mode demonstrates how a Web interface can connect browser media capabilities with a Python detection service.

4.9 Settings and Model Control

In relation to settings and Model Control, a more detailed academic interpretation is required because settings controls let users tune inference behaviour and understand the active runtime configuration. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this

reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that settings and Model Control has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

The settings panel is central to the system because it transforms the interface from a passive display into an experimental workstation. Users can adjust thresholds and observe how the results change. This is valuable for understanding the behaviour of an object detector because there is often no universally optimal threshold for every image. The interface therefore supports exploration and parameter tuning.

Model selection and device switching also make the system more flexible. If multiple model files are available, the user can compare them without editing code. If GPU support is available, the user can select it; otherwise, the interface clearly indicates that CPU is the effective device. This feedback prevents confusion and helps users understand the runtime environment.

The settings panel allows users to select a detection model, choose CPU or GPU, adjust confidence threshold, adjust IoU threshold, show or hide labels, and show the original image. These settings are sent to the backend through `PUT /api/settings`.

The backend validates and stores settings in `app_settings.json`. When the model path or device changes, the backend clears the cached model so that the next detection uses the new configuration. The backend also reports the effective device. This distinction is important because GPU selection may not be possible on every machine.

4.10 Result Visualisation and Export

In relation to result Visualisation and Export, a more detailed academic interpretation is required because visualisation and export convert backend predictions into evidence that users can inspect and preserve. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that result Visualisation and Export has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

Result visualisation is more than decoration in an inspection system. It is the mechanism through which users assess the credibility of automated predictions. The annotated preview provides spatial evidence, while the detection table provides structured evidence. Summary tiles offer a quick overview of target count and latency. Together, these elements support both rapid assessment and detailed inspection.

The export function contributes to traceability. Inspection results may need to be included in reports, compared with later results, or reviewed after a demonstration. Session-based saving ensures that outputs are not merely temporary visual states in the browser. Instead, they can be copied to a known project directory. This strengthens the practical value of the system for academic documentation and future testing.

The result area contains both visual and tabular information. The preview panel displays annotated images, selected batch outputs, video frames, or camera-frame results. The inspector panel displays target count, latency, display settings, and detection rows. Each row includes class label, class key, confidence text, and bounding box coordinates.

User input -> React upload state -> FormData request -> Python detection service -> session output -> preview URL and JSON response -> React visualisation -> optional export to `save\_data`

The export design is session-based. Image, batch, and video detections create a session ID. When the user clicks save, the frontend calls `/api/sessions/{sessionId}/save`, and the backend copies output files into the project `save_data` directory. This makes results traceable and easier to include in demonstrations or reports.

4.11 Build and Deployment

In relation to build and Deployment, a more detailed academic interpretation is required because the build approach supports both fast development and simple local production deployment. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that build and Deployment has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

The build process demonstrates that the Web interface can move beyond development mode. Vite produces static assets that are served by the Python backend, allowing the system to run through a single backend address after building. This is a practical deployment strategy for local inspection tools because it avoids maintaining a separate production Node server.

The development and production modes also support different stages of the project. During implementation, Vite's development server enables fast feedback and easier debugging. During demonstration, the built assets reduce the number of moving parts. This dual-mode arrangement supports both engineering productivity and presentation reliability.

During development, the frontend is served by Vite at `'http://127.0.0.1:5173'`, and the backend runs at `'http://127.0.0.1:8765'`. The Vite configuration proxies `'/api'` and `'/media'` to the backend, simplifying development and avoiding manual cross-origin configuration.

For production use, the frontend can be built with `'npm run build'`. The current project already contains `'web-ui/dist/index.html'`, a bundled JavaScript file, and a bundled CSS file. After building, the Python backend can host the static frontend directly. This is a practical deployment choice for a local final year project because the user only needs to start the Python backend to access the Web workstation.

Chapter 5 Testing and Evaluation

5.1 Testing Strategy

In relation to testing Strategy, a more detailed academic interpretation is required because the testing strategy evaluates the system as an integrated workflow rather than as isolated functions only. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that testing Strategy has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

The testing strategy is designed around the user workflow rather than only around isolated functions. This is appropriate because the main project contribution is an integrated workstation. A feature is successful only if the user can complete the whole sequence from

input selection to result inspection. Therefore, the tests consider interface actions, backend requests, returned data, and rendered output together.

The evaluation also distinguishes between model performance and interface performance. Model metrics indicate the capability of the detector, while functional tests indicate whether the Web workstation exposes that capability correctly. This distinction is important because a strong model can still be difficult to use if the interface is poorly designed, and a good interface cannot compensate for a completely ineffective detector.

The evaluation combines functional testing, integration testing, interface inspection, and recorded model validation metrics. This approach is appropriate because the project contribution is both a Web interface and a working integration with a detection backend. The evaluation therefore needs to answer two questions: whether the Web workstation functions correctly, and whether it is connected to a meaningful detection capability.

The tests focus on the actual implemented features: single-image detection, batch detection, video detection, camera detection, settings updates, result display, and result export. Model metrics are reported from the existing `runs/results.csv` file. The thesis does not invent additional performance values that were not recorded.

5.2 Dataset and Defect Categories

In relation to dataset and Defect Categories, a more detailed academic interpretation is required because dataset structure and class definitions determine how detection results should be labelled and interpreted. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that dataset and Defect Categories has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active

model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

The dataset statistics provide useful context for interpreting the model results and the interface design. The six categories represent different physical defect patterns, and the interface must display them in a form that users can understand. Mapping class keys to readable labels is therefore an important presentation task. A label such as `'missing_hole'` is meaningful to developers, while `'Missing Hole'` is more readable in a user interface.

The dataset also highlights the importance of future validation. The current system demonstrates a working pipeline, but further development should include more diverse PCB images and more systematic testing across lighting conditions, board layouts, and defect sizes. The Web workstation can support this future work because it already includes batch review and result export, which are useful for collecting qualitative evidence.

The dataset configuration contains six classes: missing hole, mouse bite, open circuit, short, spur, and spurious copper. The local dataset folder contains 554 training images, 551 training label files, 139 validation images, and 139 validation label files. The small difference between training images and labels should be reviewed before any further model training, but it does not prevent the Web workstation from operating with the already trained model.

The Web interface does not expose the dataset directly. Instead, the dataset is relevant because it defines the class names that appear in the detection result table. Presenting class keys and labels consistently is important for usability.

5.3 Functional Testing

In relation to functional Testing, a more detailed academic interpretation is required because functional testing verifies whether the implemented interface supports the claimed modes and user actions. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that functional Testing has direct consequences for maintainability and future extension. When a detection project is organised as a set of

unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

Functional testing verifies that the implemented modes are not independent mock-ups but connected parts of a working system. The single-image, batch, video, and camera screenshots provide visual evidence of these tests. They show that the interface changes according to the selected mode, displays returned media, and presents detection summaries. This visual evidence complements the source code and helps demonstrate that the system has been executed.

The tests also reveal practical performance characteristics. For example, latency differs between detection modes because image, batch, video, and camera inputs require different backend operations. The interface reports latency so that users can understand processing cost. This is a useful feature for inspection applications, where responsiveness may influence whether a system is suitable for online or offline use.

Table 5.1 summarises the functional tests performed or prepared for the implemented system.

Table 5.1: Functional test cases

Test ID	Feature	Expected result	Evaluation
T1	Backend status	The UI receives dependency status, settings, and defect classes.	Supported by <code>`/api/status`</code> .
T2	Model list	Available models are listed and the current model is marked.	Supported by <code>`/api/models`</code> .
T3	Settings update	Thresholds, device, labels, and original image mode are updated.	Supported by <code>`PUT /api/settings`</code> .
T4	Single-image detection	An annotated preview and detection table are returned.	Implemented through <code>`/api/detect/image`</code> .
T5	Batch	Multiple images are	Implemented through

	detection	processed and browsed through thumbnails.	`/api/detect/batch-images`.
T6	Video detection	An annotated video and sampled frame previews are returned.	Implemented through ` <code>/api/detect/video`</code> .
T7	Camera detection	Browser frames are captured and uploaded periodically.	Implemented through camera canvas capture.
T8	Result filtering	Detection rows are filtered by typed query.	Implemented with ` <code>useDeferredValue`</code> .
T9	Result export	Session output is copied into ` <code>save_data`</code> .	Implemented through ` <code>/api/sessions/{sessionId}/save`</code> .
T10	Production build	Static frontend files are generated under ` <code>web-ui/dist`</code> .	Confirmed by existing build output.

The tests show that the Web workstation covers the expected end-to-end workflow. It is not limited to displaying a static page; it communicates with the backend, sends files, receives results, and manages user interaction.

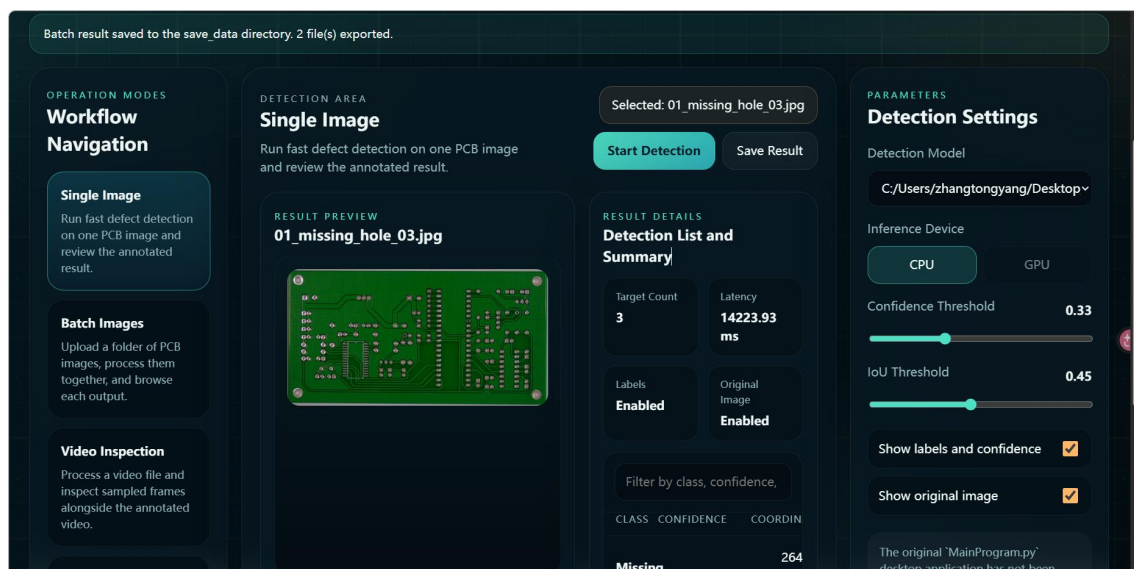


Figure 5.1 Single-image PCB defect detection result in the React Web workstation

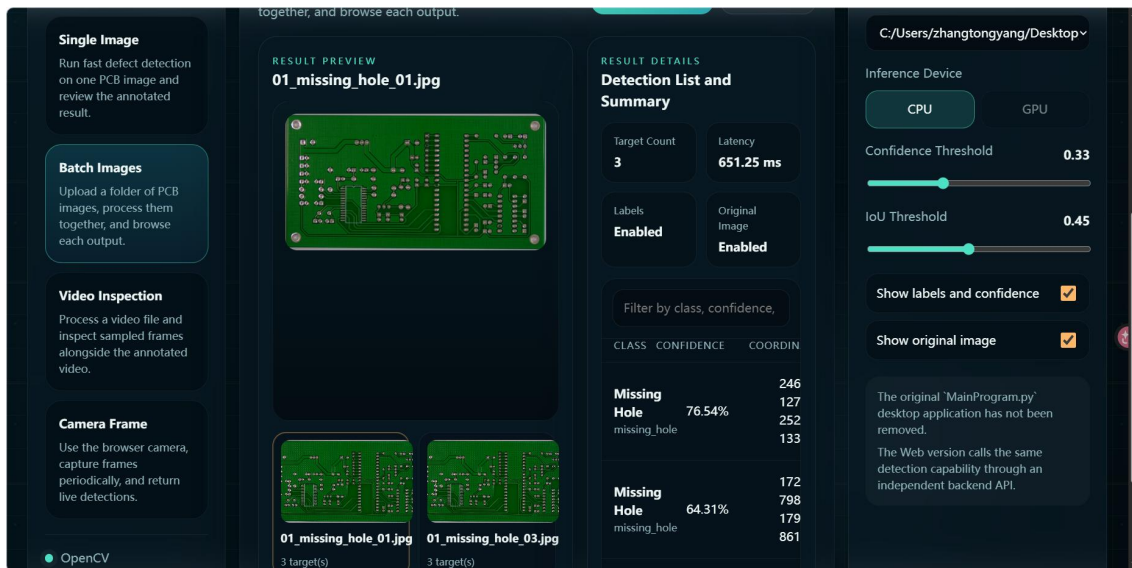


Figure 5.2 Batch image detection result in the React Web workstation

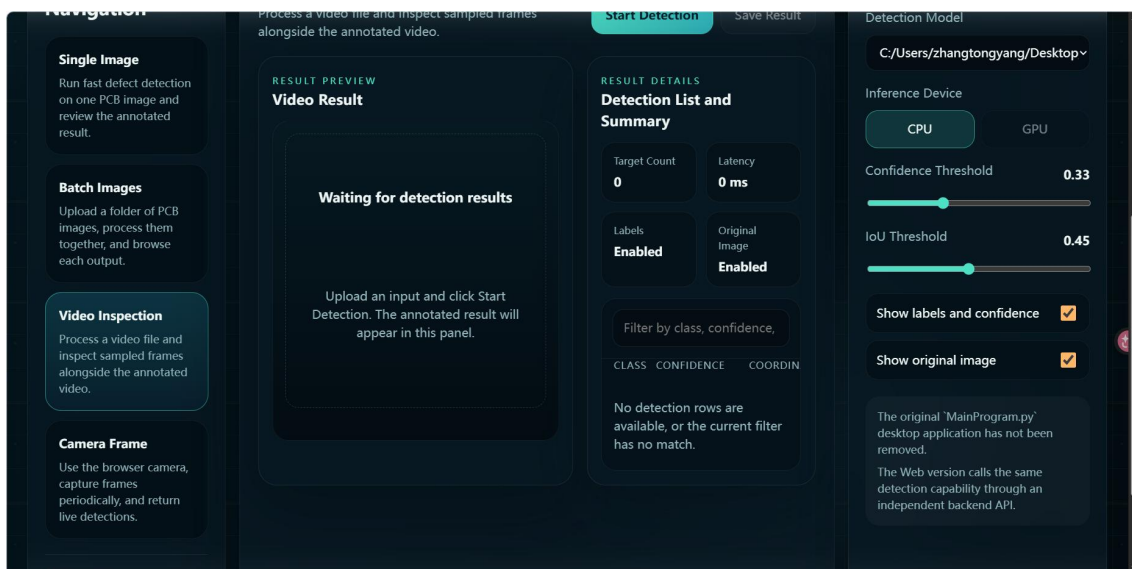


Figure 5.3 Video inspection interface in the React Web workstation

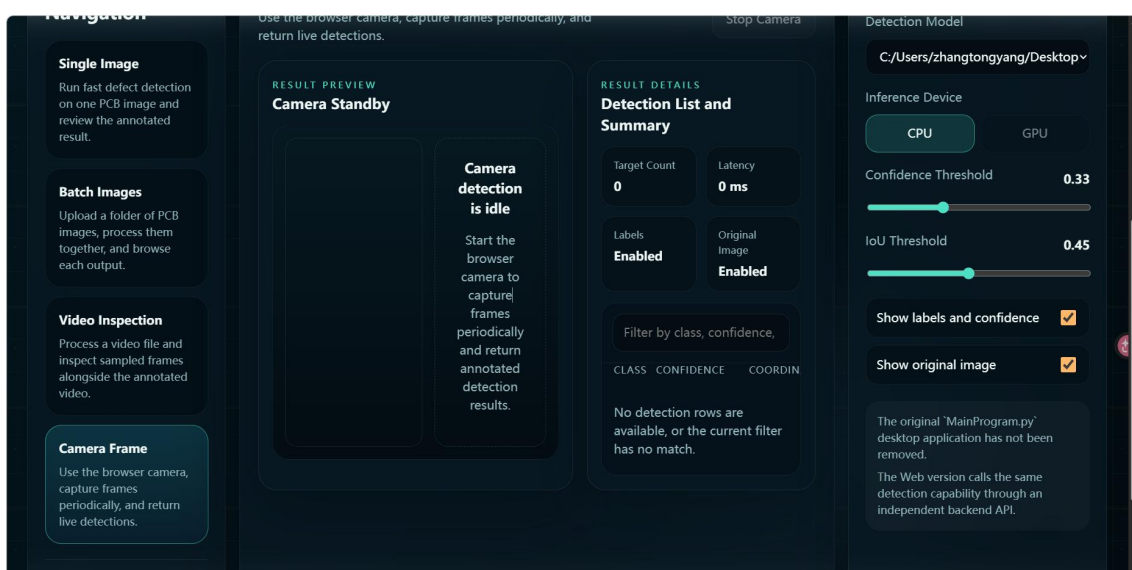


Figure 5.4 Camera-frame detection interface in the React Web workstation

5.4 Model Validation Results

In relation to model Validation Results, a more detailed academic interpretation is required because model metrics provide evidence that the frontend is connected to a meaningful detection backend. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that model Validation Results has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

The recorded validation metrics indicate that the backend model has sufficient performance to support meaningful system integration. Precision and recall show that the detector is capable of identifying a substantial proportion of relevant defects while limiting incorrect detections. The mAP metrics provide a more detailed view of localisation and classification performance across thresholds. These values support the claim that the frontend is connected to a functional detection backend.

At the same time, the lower mAP@0.5:0.95 value suggests that localisation quality remains an area for improvement. This is important because PCB defects may be small and precise localisation can be difficult. The Web workstation helps expose this issue because users can visually inspect bounding boxes and compare them with the actual defect location. Thus, the interface can also be used as a qualitative evaluation tool.

The model validation record in `runs/results.csv` contains 170 epochs. The final recorded epoch reports the values shown in Table 5.2.

Table 5.2: Recorded model validation results

Metric	Final recorded value
Precision	0.95914
Recall	0.88724
mAP@0.5	0.92431
mAP@0.5:0.95	0.46904
Validation box loss	1.7774
Validation classification loss	0.8101
Validation DFL loss	0.92918

These values indicate that the detector has a strong ability to identify many PCB defects at the looser mAP@0.5 threshold. The lower mAP@0.5:0.95 value suggests that stricter localisation quality remains more challenging. This is a common pattern in object detection tasks where small or subtle objects require precise bounding boxes. For the Web workstation, these metrics provide evidence that the backend model is meaningful enough to support interface evaluation.

5.5 Interface and Workflow Evaluation

In relation to interface and Workflow Evaluation, a more detailed academic interpretation is required because workflow evaluation considers whether the system is understandable, efficient, and suitable for repeated inspection tasks. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that interface and Workflow Evaluation has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active

model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

The interface evaluation focuses on whether the workstation supports a coherent inspection workflow. The screenshots show that the user can move between modes while retaining a consistent layout. The repeated structure of navigation, preview, details, and settings reduces learning effort. This consistency is particularly important for a system that supports multiple media types, because each mode could otherwise feel like a separate tool.

The visual hierarchy also supports task focus. Important actions such as starting detection are presented as primary buttons, while save actions are secondary. Runtime information and model configuration are visible but do not dominate the result preview. This balance helps the user concentrate on inspection results while still retaining access to configuration and status information.

The main usability improvement of the Web workstation is the consolidation of multiple workflows into one page. A user can select a mode, upload input, adjust parameters, inspect results, and save outputs without switching tools. The left mode rail reduces confusion between image, batch, video, and camera workflows. The central preview focuses attention on the current result. The right settings panel keeps model and threshold controls visible.

The result table improves interpretability by exposing the details behind each annotation. The table is especially useful when several defects appear in one image or frame. The search filter helps users locate rows by class name, confidence value, or coordinate text. This is a practical frontend contribution because it turns raw detection output into an inspectable record.

5.6 Technical Evaluation

In relation to technical Evaluation, a more detailed academic interpretation is required because technical evaluation reviews the maintainability, integration quality, and appropriateness of the implementation choices. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that technical Evaluation has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

Technically, the project demonstrates effective integration of browser APIs, React state, typed request handling, and Python backend services. The implementation is not overly complex, but it addresses real engineering concerns such as asynchronous detection, media upload, camera cleanup, and result rendering. This makes the system more substantial than a static frontend page.

The use of a small technology stack is a strength rather than a limitation for this project. By avoiding unnecessary routing, global stores, and backend frameworks, the implementation remains transparent and suitable for local deployment. The resulting codebase is easier to explain, test, and extend. This is consistent with good engineering practice for a focused final year project.

The frontend implementation is lightweight and appropriate for the project scope. It avoids unnecessary frameworks while still using TypeScript for API structure. The API layer reduces repeated fetch logic. The use of `'startTransition'` and `'useDeferredValue'` demonstrates attention to responsiveness. The use of `'useRef'` for camera stream and interval state prevents avoidable re-rendering.

The backend integration is also practical. The standard-library HTTP server is sufficient for local use and coursework demonstration. It supports CORS headers, static frontend hosting, range requests for media, multipart form parsing, and JSON responses. The detection service separates HTTP handling from detection logic, which improves maintainability.

5.7 Limitations

In relation to limitations, a more detailed academic interpretation is required because limitations define the boundary between the completed prototype and a production-grade inspection platform. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this

reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that limitations has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

The limitations should be understood as boundaries of the current prototype rather than failures of the implementation. The system demonstrates Web-based interaction with a local detection backend, but it does not yet include enterprise deployment features such as authentication, audit logs, multi-user concurrency control, or secure remote access. These features would be necessary in a production manufacturing environment.

The evaluation is also limited by the available evidence. Functional screenshots and recorded model metrics are useful, but a stronger evaluation would include structured user feedback, repeated timing measurements, and comparison with the original desktop workflow. Future work should therefore treat the current workstation as a foundation for more systematic usability and performance studies.

Several limitations remain. First, the system is mainly designed for local deployment and does not include user authentication, role management, or secure multi-user access. Second, video detection can be slow because every frame may require model inference. Third, camera mode depends on browser permissions and hardware availability. Fourth, the UI evaluation was primarily functional; a formal user study with multiple participants was not conducted. Fifth, the training dataset has a small mismatch between training images and label files, which should be checked before retraining.

These limitations do not undermine the main contribution, but they show where future work should focus if the system is extended beyond a final year project prototype.

Chapter 6 Conclusion and Future Work

6.1 Conclusion

In relation to conclusion, a more detailed academic interpretation is required because the conclusion relates the implemented system back to the original aim and assesses the value of the Web workstation. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that conclusion has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

The final system demonstrates that a React-based Web interface can make a PCB defect detection pipeline more accessible and operationally coherent. By integrating multiple input modes, parameter controls, result previews, detection tables, and export functions, the project provides a complete interaction layer around the backend model. This confirms the central claim of the dissertation: frontend engineering is a significant part of practical intelligent inspection systems.

This project designed and implemented a React-based Web workstation for PCB defect detection. The workstation provides a unified browser interface for operating an existing Python and YOLO detection backend. It supports single-image, batch-image, video, and camera-frame detection. It also provides model selection, device switching, threshold adjustment, display options, result preview, detection table filtering, and session-based export. The project demonstrates that frontend engineering is an important part of intelligent inspection systems. A model may perform detection, but a user still needs a clear and reliable workflow for operating the model and interpreting the outputs. The implemented Web

workstation improves this workflow by organising input selection, parameter configuration, result visualisation, and export into one coordinated interface.

6.2 Reflection

In relation to reflection, a more detailed academic interpretation is required because reflection identifies what was learned about frontend engineering, backend integration, and practical intelligent inspection systems. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that reflection has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

Reflecting on the project, the most important lesson is that system usefulness depends on the alignment between backend capability and frontend workflow. A technically capable detector can remain difficult to use if the interface does not expose its settings and results clearly. Conversely, a well-designed interface must respect the constraints of the backend, such as processing time, file formats, and model availability. The project required continuous attention to both sides of this relationship.

The project required both technical implementation and design judgement. On the frontend side, the most important challenge was managing several workflows without making the interface confusing. The solution was to use a mode-based layout while keeping settings and results visible. On the integration side, the challenge was to connect browser file upload, camera capture, and video processing to a Python detection service. The solution was to define a small but clear set of HTTP APIs and TypeScript result types.

The work also showed the importance of not overcomplicating a system. The Web application does not need React Router or Redux because the scope is a single-page workstation. The backend does not need Flask or FastAPI for local demonstration because the standard library server is sufficient. These decisions keep the project understandable and easier to maintain.

6.3 Future Work

In relation to future Work, a more detailed academic interpretation is required because future work indicates how the prototype could evolve toward stronger evaluation, deployment, and inspection analytics. A final year project should not only describe what was implemented, but should also explain why the implementation is appropriate for the problem context. For this reason, the discussion treats the Web workstation as a socio-technical artefact: it combines technical mechanisms, such as HTTP APIs and typed data structures, with the human need to operate an inspection system in a clear and repeatable manner. This perspective strengthens the dissertation because it connects software design decisions with real inspection requirements rather than presenting the interface as a purely visual addition.

The implemented system also shows that future Work has direct consequences for maintainability and future extension. When a detection project is organised as a set of unrelated scripts, each new feature usually requires additional manual coordination by the user. By contrast, the React workstation places related operations into a consistent interaction model, which makes the system easier to operate and easier to explain. This is particularly important in an assessment context because the evaluator can observe the active model, settings, input mode, output preview, and result table within the same interface. The resulting software is therefore more transparent than a backend-only implementation.

Future work should also consider stronger evidence collection. For example, the workstation could store each detection session in a small database with input metadata, settings, latency, and exported files. This would support longitudinal comparison of models and thresholds. A reporting page could then summarise defect counts, common defect types, and processing times. Such features would make the system more useful for practical inspection analysis and for future academic evaluation.

Future work can improve the system in several directions. First, a user login system and history database could be added so that multiple users can store and compare inspection sessions. Second, a dashboard could provide charts for defect frequencies, class distribution,

confidence distribution, and processing time. Third, the video pipeline could be optimised through frame sampling, asynchronous processing, or GPU batching. Fourth, camera detection could be improved with real-time streaming rather than periodic frame upload. Fifth, the model management page could be extended to support model upload, model comparison, and training logs. Sixth, a formal user study could evaluate whether the Web workstation improves usability compared with command-line or desktop-only operation.

Overall, the project provides a functional and extensible foundation for Web-based PCB defect inspection. It verifies that a React interface can make an intelligent detection backend more accessible, visible, and practical for demonstration and future development.

Chapter 7 References

- [1] M. Moganti, F. Ercal, C. H. Dagli, and S. Tsunekawa, "Automatic PCB inspection algorithms: A survey," *Computer Vision and Image Understanding*, vol. 63, no. 2, pp. 287-313, 1996, doi: 10.1006/cviu.1996.0020.
- [2] W. Huang and P. Wei, "A PCB dataset for defects detection and classification," *arXiv preprint arXiv:1901.08204*, 2019.
- [3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779-788, doi: 10.1109/CVPR.2016.91.
- [4] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 7464-7475.
- [5] V. A. Adibhatla, H.-C. Chih, C.-C. Hsu, J. Cheng, M. F. Abbod, and J.-S. Shieh, "Defect detection in printed circuit boards using You-Only-Look-Once convolutional neural networks," *Electronics*, vol. 9, no. 9, Art. no. 1547, 2020, doi: 10.3390/electronics9091547.
- [6] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, vol. 25, no. 11, pp. 120-125, 2000.
- [7] M. Madsen, O. Lhotak, and F. Tip, "A semantics for the essence of React," in *Proc. 34th European Conf. Object-Oriented Programming (ECOOP)*, 2020, pp. 12:1-12:26, doi: 10.4230/LIPIcs.ECOOP.2020.12.
- [8] G. M. Bierman, M. Abadi, and M. Torgersen, "Understanding TypeScript," in *ECOOP 2014 - Object-Oriented Programming, Lecture Notes in Computer Science*, vol. 8586. Berlin, Germany: Springer, 2014, pp. 257-281, doi: 10.1007/978-3-662-44202-9_11.

- [9] J. Rack and C.-A. Staicu, "Jack-in-the-box: An empirical study of JavaScript bundling on the Web and its security implications," in Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS), 2023, pp. 3198-3212, doi: 10.1145/3576915.3623140.
- [10] S. Bae, H. Cho, I. Lim, and S. Ryu, "SAFEWAPI: Web API misuse detector for web applications," in Proc. 22nd ACM SIGSOFT Int. Symp. Foundations of Software Engineering (FSE), 2014, pp. 507-517, doi: 10.1145/2635868.2635916.
- [11] B. Feher, L. Sidi, A. Shabtai, and R. Puzis, "The security of WebRTC," arXiv preprint arXiv:1601.00184, 2016.
- [12] P. Virtanen et al., "SciPy 1.0: Fundamental algorithms for scientific computing in Python," Nature Methods, vol. 17, pp. 261-272, 2020, doi: 10.1038/s41592-019-0686-2.

Chapter 8 Appendix A. Operation Guide

8.1 Appendix A.1 Development Mode

Start the Python backend from the project root:

```
`python -m web_backend`
```

Start the React frontend from the `web-ui` directory:

```
`npm run dev`
```

Then open:

```
`http://127.0.0.1:5173`
```

The Vite server proxies `/api` and `/media` requests to `http://127.0.0.1:8765`.

8.2 Appendix A.2 Production Mode

Build the frontend:

```
`npm run build`
```

Start the backend:

```
`python -m web_backend`
```

Then open:

```
`http://127.0.0.1:8765`
```

In this mode, the Python backend serves the built frontend files from `web-ui/dist`.

Chapter 9 Appendix B. API Summary

The API contract is intentionally small. Status and configuration are handled through GET and PUT endpoints. Detection inputs are uploaded through multipart form data. Results are returned as JSON and media paths. Image, batch, and video results are associated with session IDs, which makes later export possible.

The most important response object is the detection item. It contains the object index, class ID, class key, class label, confidence, formatted confidence text, and bounding box coordinates. This common structure allows the same detection table component to display image, batch, video, and camera results.